PredomicsApp

Codebase Documentation

A full-stack web application for interpretable predictive modeling of omics data using sparse discrete linear classifiers

Version: 0.1.0

Generated: February 18, 2026

Authors: Edi Prifti & contributors

License: Proprietary / Research

Repository: predomicsapp-web

Table of Contents

1. Project Overview
What is PredomicsApp? · Key Statistics

2. Architecture & Technology Stack
System Diagram · Frontend · Backend · ML Engine

3. Project Structure
Root Layout · Frontend Tree · Backend Tree

4. Frontend — Vue 3 SPA
Views · Components · Stores · Composables · Internationalization

5. Backend — FastAPI
API Routers · Services · Core Modules

6. API Reference
Auth · Projects · Datasets · Analysis · Data Explore · Export

7. Database Schema
Tables · Relationships

8. Algorithms & Methods
Model Languages · Optimization · Stability · Ecosystem

9. Visualization System
Results Dashboard · Plotly Charts · Dark/Light Theming

10. Deployment & Configuration
Docker · Kubernetes · Environment

11. Testing
Backend Tests · Frontend Tests

1. Project Overview

What is PredomicsApp?

PredomicsApp is a production-grade, full-stack web application that provides an interactive interface for **gpredomics**, a Rust-accelerated engine for building interpretable predictive models from high-dimensional omics data (metagenomics, metabolomics, transcriptomics).

The application implements **sparse discrete linear classifiers** — models that use only a handful of features with integer coefficients (+1, -1, +2, etc.) — making them inherently interpretable and deployable as simple biomarker signatures. The optimization is performed via genetic algorithms (GA), beam search, or MCMC, producing a *population* of near-optimal models rather than a single solution.

> **Key Innovation:** Rather than returning a single model, PredomicsApp explores the *Family of Best Models* (FBM) — the statistically defined population of all models whose performance is not significantly worse than the best. This enables ecosystem-level analysis of feature co-selection, stability, and biological relevance.

Key Statistics

34,056 LINES OF CODE	**90+** API ENDPOINTS	**15** DATABASE TABLES
18 FRONTEND VIEWS	**18** API ROUTERS	**15** BACKEND SERVICES
2 LANGUAGES (EN/FR)	**215+** TEST CASES	**5** EXPORT FORMATS

2. Architecture & Technology Stack

System Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          NGINX Reverse Proxy                         │
│                       (SSL/TLS, rate limiting)                       │
└──────────────────────────────────────────────────────────────────────┘
              │                                │
              ▼                                ▼
┌──────────────────────────┐     ┌──────────────────────────┐
│   Vue 3 SPA Frontend     │     │   FastAPI Backend        │
│                          │     │                          │
│  • Plotly.js charts      │     │  • REST API (90+ eps)    │
│  • Pinia state           │ ←→  │  • WebSocket progress    │
│  • Vue Router            │     │  • JWT + API key auth    │
│  • vue-i18n (EN/FR)      │     │  • SQLAlchemy async ORM  │
│  • Vite build            │     │  • ReportLab PDF gen     │
└──────────────────────────┘     └──────────────────────────┘
                                              │
                                              ▼
┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────────────┐
│   PostgreSQL             │  │  gpredomicspy            │  │  File Storage            │
│   (async via asyncpg)    │  │  (Rust + Python)         │  │  (TSV, JSON)             │
│                          │  │                          │  │                          │
│  • Users, Projects       │  │  • GA / Beam /           │  │  • X/Y train             │
│  • Jobs, Datasets        │  │    MCMC search           │  │  • Results               │
│  • Shares, Comments      │  │  • Rust-accel.           │  │  • Exports               │
│  • Audit, Webhooks       │  │    filtering             │  │  • Backups               │
└──────────────────────────┘  └──────────────────────────┘  └──────────────────────────┘
```

Frontend Stack

Technology	Version	Purpose
Vue 3	3.4+	Reactive UI framework (Composition API with `<script setup>`)
Pinia	2.1+	State management (auth, projects, config, datasets, theme)
Vue Router	4.2+	Client-side routing with guards and lazy loading
Plotly.js	2.27+	Interactive charts (30+ chart types across the app)
vue-i18n	9.8+	Internationalization — English and French
Axios	1.6+	HTTP client with JWT interceptors
Vite	5.0+	Build tool with HMR and code splitting
Vitest	1.0+	Unit testing with jsdom

Backend Stack

Technology	Version	Purpose
FastAPI	0.104+	Async web framework with auto-generated OpenAPI docs
SQLAlchemy	2.0+	Async ORM with PostgreSQL/SQLite
asyncpg	0.29+	High-performance async PostgreSQL driver
Pydantic	2.0+	Data validation and serialization
python-jose	3.3+	JWT token generation and validation
ReportLab	4.0+	PDF biomarker report generation
NetworkX	3.0+	Graph analysis (Louvain community detection, centrality)
scikit-learn	1.0+	t-SNE, SHAP, MDA feature importance
umap-learn	0.5+	UMAP dimensionality reduction
scipy	1.10+	Statistical tests (Wilcoxon, Spearman, hypergeometric)

ML Engine — gpredomicspy

The core ML engine is **gpredomicspy**, a hybrid Rust + Python package that implements:

- **Feature filtering:** Rust-accelerated prevalence and significance filtering (Wilcoxon, Student's t, Bayesian Fisher)
- **Optimization:** Genetic Algorithm (GA), Beam Search, MCMC for sparse model search
- **Model languages:** Binary (±1), Ternary (0,±1), Ratio, Power-of-2 encodings
- **Cross-validation:** Stratified k-fold with train/test tracking
- **Evaluation:** AUC, accuracy, sensitivity, specificity, fit (composite metric)

3. Project Structure

Root Layout

```
predomicsapp-web/
├── backend/              — FastAPI application (Python)
│   ├── app/              — Main package
│   │   ├── main.py        — App entry point, migrations, startup
│   │   ├── core/          — Config, DB, security, deps
│   │   ├── models/        — SQLAlchemy models & Pydantic schemas
│   │   ├── routers/       — 18 API endpoint modules
│   │   └── services/      — 15 business logic modules
│   ├── tests/             — pytest test suite
│   └── pyproject.toml     — Python project metadata & deps
├── frontend/             — Vue 3 SPA (JavaScript)
│   ├── src/
│   │   ├── views/         — 18 page-level components
│   │   ├── components/    — 15 reusable components
│   │   ├── stores/        — 5 Pinia stores
│   │   ├── composables/   — 5 Vue 3 composables
│   │   ├── i18n/          — EN/FR translation files
│   │   ├── utils/         — Helper functions
│   │   ├── data/          — Parameter definitions
│   │   ├── App.vue        — Root component
│   │   ├── router.js      — Vue Router config
│   │   └── main.js        — Entry point
│   ├── public/            — Static assets (logos, favicons)
│   └── package.json       — npm dependencies
├── helm/                 — Kubernetes Helm charts
├── nginx/                — NGINX reverse proxy config
├── scripts/              — Deployment & utility scripts
├── docs/                 — Screenshots & Playwright automation
├── docker-compose.yml     — Development stack
├── Dockerfile             — Multi-stage build
└── README.md              — Project README
```

4. Frontend — Vue 3 SPA

Views (Page Components)

View	Route	Description
`HomeView.vue`	`/`	Landing page with animated workflow, feature showcase, and getting started guide
`LoginView.vue`	`/login`	Authentication form (login + register toggle)
`DashboardView.vue`	`/dashboard`	Global stats, recent activity, quick actions
`ProjectsView.vue`	`/projects`	Project listing with search, sort, archive filter
`ProjectDashboard.vue`	`/project/:id`	Project layout with Data / Parameters / Results tabs
`DataTab.vue`	`/project/:id/data`	Dataset upload, preview, QC stats, PCoA/t-SNE/UMAP visualization
`ParametersTab.vue`	`/project/:id/params`	Algorithm configuration with validation, batch setup, job launch
`ResultsTab.vue`	`/project/:id/results`	**Main results dashboard** — 10 sub-tabs with 30+ chart types (182 KB, largest file)
`MetaAnalysisView.vue`	`/meta-analysis`	Cross-cohort comparison: feature overlap, concordance, meta-AUC
`DatasetLibrary.vue`	`/datasets`	Shared dataset management with versioning, tags, preview
`AdminView.vue`	`/admin`	Admin panel: user management, defaults, backups, audit log, SciTQ config
`ProfileView.vue`	`/profile`	User settings, API keys, theme preference
`PublicShareView.vue`	`/public/:token`	Public read-only project view (no authentication required)
`CreditsView.vue`	`/credits`	Credits, citations, funding acknowledgments

Reusable Components

Component	Description
`ConsolePanel.vue`	Real-time job log streaming via WebSocket with ANSI color support
`ValidateModal.vue`	External validation: upload test data, run deployed model, show confusion matrix
`ParamSection.vue`	Collapsible parameter group with type-aware inputs (range, select, checkbox)
`ShareModal.vue`	User-to-user project sharing with viewer/editor role selection
`PublicShareModal.vue`	Generate public read-only links with optional expiry
`CommentsSidebar.vue`	Threaded comment sidebar for project-level discussion
`OnboardingTour.vue`	First-use walkthrough overlay highlighting key UI elements
`EcosystemTab.vue`	Species co-abundance network with 4 layouts, Louvain modules, taxonomy coloring
`StabilityTab.vue`	Feature stability indices (Kuncheva, Tanimoto) across sparsity levels
`SvgIcon.vue`	Inline SVG icon renderer (20+ icons)

Pinia Stores

Store	State
`auth.js`	User session, JWT token, login/logout, API key management
`project.js`	Current project, job list, active job, console state, data filters
`config.js`	Parameter form state, defaults, `loadFromJobConfig()`
`dataset.js`	Dataset list, upload state, file metadata
`theme.js`	Dark/light mode toggle with localStorage persistence

Vue 3 Composables

Composable	Purpose
`useChartTheme.js`	Plotly theming — `chartColors()` , `chartLayout()` , `featureLabel()`
`useShapValues.js`	SHAP matrix computation for beeswarm, force, and dependence plots
`useModelBasket.js`	Persistent model bookmarking (localStorage per project, max 50 models)
`useDebugMode.js`	Debug mode toggle for timing flamegraphs and verbose output
`useToast.js`	Toast notification system (success/error/info)

Internationalization

Full English and French translations via `vue-i18n` . Translation files at `src/i18n/locales/en.json` and `fr.json` contain 400+ keys covering all UI strings. Language toggle in the header bar with flag icons.

5. Backend — FastAPI

API Routers (18 modules)

Router	Prefix	Endpoints	Description
`auth.py`	`/api/auth`	11	Register, login, profile, password reset, API keys
`projects.py`	`/api/projects`	7	CRUD, archive, dataset attachment
`datasets.py`	`/api/datasets`	16	Upload, preview, versioning, tags, file management
`analysis.py`	`/api/analysis`	15	Job launch, batch runs, results, stability, validation
`data_explore.py`	`/api/data-explore`	9	Feature stats, distributions, PCoA/t-SNE/UMAP, co-abundance
`export.py`	`/api/export`	5	CSV, HTML report, PDF, JSON, Python/R notebooks
`predict.py`	`/api/predict`	1	Deploy model as prediction API
`sharing.py`	`/api/sharing`	5	User-to-user project sharing (viewer/editor)
`public.py`	`/api`	5	Public share links (no auth required)
`comments.py`	`/api/comments`	4	Threaded project comments
`dashboard.py`	`/api/dashboard`	1	Global activity statistics
`admin.py`	`/api/admin`	13	User admin, defaults, backups, audit, SciTQ
`templates.py`	`/api/templates`	5	Parameter template CRUD
`webhook_router.py`	`/api/webhooks`	4	Webhook registration and testing
`websocket.py`	`/ws`	1	Real-time job progress streaming
`samples.py`	`/api/samples`	2	Demo dataset loader

`health.py`	`/api`	1	Health check endpoint
`meta_analysis.py`	`/api/meta-analysis`	2	Cross-cohort job comparison

Business Logic Services (15 modules)

Service	Description
`engine.py`	gpredomicspy integration — wraps Rust ML engine calls
`worker.py`	Background job runner — subprocess-based with progress tracking and WebSocket updates
`storage.py`	File storage — save/load job results, manage project directories
`data_analysis.py`	Data exploration — filtering, PCoA, t-SNE, UMAP, distributions, feature abundance
`coabundance.py`	Co-abundance network — Spearman correlations, Louvain communities, taxonomy coloring
`stability.py`	Stability indices — Kuncheva, Tanimoto, CW_rel across sparsity levels
`prediction.py`	Model deployment — apply saved model to new samples via API
`pdf_report.py`	PDF generation — ReportLab-based biomarker report with charts and tables
`msp_annotations.py`	MSP taxonomy — fetch species/phylum/family from biobanks.gmt.bio with caching
`taxonomy_colors.py`	Hierarchical taxonomy color assignment (SCAPIS-inspired phylum → family shading)
`audit.py`	Audit logging — track user actions with IP, user agent, details
`backup.py`	Backup/restore — database + file system snapshots
`email.py`	SMTP email — password reset, notifications (optional dependency)
`webhooks.py`	Webhook delivery — HMAC-signed HTTP callbacks on job completion
`scitq_client.py`	SciTQ integration — distributed task queue for cluster execution

Core Modules

Module	Purpose
`config.py`	Pydantic Settings — all app configuration from environment variables
`database.py`	Async SQLAlchemy engine and session factory
`deps.py`	FastAPI dependencies — `get_current_user`, `get_project_with_access`
`security.py`	JWT creation/verification, password hashing (bcrypt)
`cache.py`	TTL-based in-memory cache for expensive computations
`errors.py`	Custom exception handlers and error formatting
`rate_limit.py`	slowapi rate limiting configuration (per-user, per-IP)

6. API Reference

The full interactive API documentation is available at `/api/docs` (Swagger UI) and `/api/redoc` (ReDoc) when the server is running. Below is a summary of the key endpoint groups.

Authentication (`/api/auth`)

Method	Path	Description
POST	`/register`	Create new user account
POST	`/login`	Authenticate and receive JWT token
GET	`/me`	Get current user profile
PUT	`/me`	Update user profile
PUT	`/me/password`	Change password
POST	`/forgot-password`	Request password reset email
POST	`/reset-password`	Reset password with token
POST	`/api-keys`	Create API key for programmatic access
GET	`/api-keys`	List user's API keys
DELETE	`/api-keys/{key_id}`	Revoke API key

Projects (`/api/projects`)

Method	Path	Description
POST	`/`	Create project
GET	`/`	List user's projects
GET	`/{id}`	Get project details
PATCH	`/{id}`	Update project name/description
DELETE	`/{id}`	Delete project and all jobs
POST	`/{id}/archive`	Toggle archive status
POST	`/{id}/datasets`	Attach dataset to project

Datasets (`/api/datasets`)

Method	Path	Description
POST	`/`	Create dataset with metadata
GET	`/`	List datasets
POST	`/{id}/files`	Upload data file (xtrain, ytrain, xtest, ytest)
GET	`/{id}/files/{fid}/preview`	Preview file contents (first N rows)
POST	`/{id}/scan`	Auto-detect file roles and dimensions
PATCH	`/{id}/tags`	Update dataset tags
GET	`/{id}/versions`	List version history
POST	`/{id}/versions/{vid}/restore`	Restore to previous version
POST	`/{id}/assign/{pid}`	Assign dataset to project

Analysis (`/api/analysis`)

Method	Path	Description
POST	`/{pid}/run`	Launch single analysis job
POST	`/{pid}/batch`	Launch batch parameter sweep (cartesian product, max 50)
GET	`/{pid}/jobs`	List all jobs for project
GET	`/{pid}/jobs/{jid}`	Job summary (status, metrics)
GET	`/{pid}/jobs/{jid}/detail`	Job detail with best individual
GET	`/{pid}/jobs/{jid}/results`	Full results (population, generation tracking, jury)
GET	`/{pid}/jobs/{jid}/stability`	Pre-computed stability analysis
POST	`/{pid}/jobs/{jid}/validate`	External validation with uploaded test data
POST	`/{pid}/jobs/{jid}/rerun`	Re-run job with same config
GET	`/{pid}/jobs/{jid}/logs`	Console output log
GET	`/{pid}/jobs/{jid}/progress`	Incremental progress (generation, AUC)
DELETE	`/{pid}/jobs/{jid}`	Delete job and results

Data Exploration (`/api/data-explore`)

Method	Path	Description
GET	`/{pid}/summary`	Data dimensions, class distribution
GET	`/{pid}/feature-stats`	Per-feature statistics (prevalence, significance, mean, std)
GET	`/{pid}/distributions`	Pre-binned histograms for prevalence and SD
GET	`/{pid}/feature-abundance`	Boxplot stats per class for selected features
GET	`/{pid}/barcode-data`	Raw heatmap matrix (features × samples)
GET	`/{pid}/ordination`	PCoA / t-SNE / UMAP with PERMANOVA and confidence ellipses
GET	`/{pid}/coabundance-network`	Co-abundance ecosystem network with Louvain modules
POST	`/msp-annotations`	MSP taxonomy lookup from biobanks.gmt.bio

Export (`/api/export`)

Method	Path	Description
GET	`/{pid}/jobs/{jid}/csv?section=...`	CSV: best_model, population, generation_tracking, jury_predictions
GET	`/{pid}/jobs/{jid}/report`	HTML biomarker report
GET	`/{pid}/jobs/{jid}/pdf`	PDF biomarker report (ReportLab)
GET	`/{pid}/jobs/{jid}/json`	Full JSON results download
GET	`/{pid}/jobs/{jid}/notebook?lang=python|r`	Jupyter notebook (.ipynb) or R Markdown (.Rmd)

7. Database Schema

Tables (15)

Table	Key Columns	Purpose
`users`	id, email, username, hashed_password, is_admin, is_active	User accounts
`projects`	id, name, description, owner_id, archived, created_at	Analysis projects
`datasets`	id, name, owner_id, tags, notes, archived, class_names	Dataset metadata
`dataset_files`	id, dataset_id, role, filename, disk_path, n_rows, n_cols	Individual data files (xtrain, ytrain, etc.)
`project_datasets`	project_id, dataset_id	Many-to-many: projects ↔ datasets
`jobs`	id, project_id, user_id, name, status, config, best_auc, best_k, config_hash	Analysis jobs
`project_shares`	id, project_id, user_id, role (viewer/editor)	User-to-user sharing
`public_shares`	id, project_id, token, expires_at	Public read-only links
`project_comments`	id, project_id, user_id, content, parent_id	Threaded comments
`api_keys`	id, user_id, name, key_hash, last_used_at	Programmatic API access
`webhooks`	id, user_id, url, events, secret_hash	Event-driven HTTP callbacks
`audit_log`	id, user_id, action, resource_type, resource_id, ip_address	Action audit trail
`password_reset_tokens`	id, user_id, token_hash, expires_at, used	Password reset flow
`dataset_versions`	id, dataset_id, version_number, snapshot_path	Dataset version history
`schema_version`	version, applied_at	Database migration tracking

Key Relationships

```
User ──1:N──► Project ──1:N──► Job
  │                │
  │                ├──M:N──► Dataset ──1:N──► DatasetFile
  │                │                    │
  │                ├──1:N──► ProjectShare    └──1:N──► DatasetVersion
  │                ├──1:N──► PublicShare
  │                └──1:N──► ProjectComment
  │
  ├──1:N──► ApiKey
  ├──1:N──► Webhook
  └──1:N──► AuditLog
```

8. Algorithms & Methods

Model Languages

PredomicsApp implements **sparse discrete linear classifiers** with four encoding schemes:

Language	Coefficients	Formula	Interpretability
Binary	+1, -1	score = Σ(±1 × feature_i)	Highest — simple presence/absence voting
Ternary	0, +1, -1	score = Σ(coef × feature_i)	High — allows feature exclusion
Ratio	log-ratio pairs	score = Σ log(feature_i / feature_j)	Medium — relative abundance ratios
Power-of-2	±1, ±2, ±4...	score = Σ(2^k × feature_i)	Medium — integer-weighted linear sum

Optimization Algorithms

- **Genetic Algorithm (GA):** Population-based evolutionary search with crossover, mutation, and tournament selection. Default method with best exploration of the model space.
- **Beam Search:** Deterministic breadth-first search expanding top-k candidates at each sparsity level. Faster but less exploratory.
- **MCMC (Markov Chain Monte Carlo):** Stochastic sampling of the model space with Metropolis-Hastings acceptance criterion.

Family of Best Models (FBM) & Stability

The **FBM** is the statistically defined population of all models whose fitness is not significantly worse than the best. Two CI methods are available:

- **Standard Wald CI:** threshold = $r - z \times \sqrt{r(1-r)/n}$
- **Blaise CI:** threshold = $r - (0.5/n + z \times \sqrt{r(1-r)/n})$ — adds continuity correction

Stability is measured across sparsity levels using:

- **Kuncheva index:** Pairwise overlap consistency of feature sets
- **Tanimoto similarity:** Jaccard-like coefficient for feature set comparison
- **CW_rel:** Relative consistency-weighted index

Ecosystem & Network Analysis

The co-abundance network analysis pipeline:

1. **Prevalence filtering:** Remove features below threshold (default 30%)
2. **Spearman correlations:** Matrix-wise computation ($O(n^2)$ for feature pairs)
3. **Edge filtering:** Keep pairs with $|\rho| \geq$ threshold (default 0.3)
4. **Community detection:** Louvain algorithm (NetworkX, seed=42 for reproducibility)
5. **Node metrics:** Degree, betweenness centrality, class enrichment
6. **Taxonomy annotation:** MSP lookup from biobanks.gmt.bio with hierarchical coloring
7. **Co-presence analysis:** Hypergeometric test for pairwise feature co-occurrence in FBM

9. Visualization System

Results Dashboard Sub-Tabs

Tab	Charts
Summary	AUC convergence, feature count evolution, fit evolution over generations
Best Model	Radar chart, coefficient bar, MDA importance, waterfall, per-sample heatmap, SHAP (beeswarm, force, dependence)
Population	Composition by language/type, AUC violin by language, feature prevalence, feature heatmap, PCoA/t-SNE/UMAP ordination with PERMANOVA
Jury	Expert comparison, concordance, confusion matrices (train/test), vote matrix, sample predictions, FBM experts
Comparative	Multi-job metrics comparison, convergence overlay, feature overlap heatmap, config diff
Co-Presence	Feature prevalence bar, co-occurrence heatmap, network visualization, stats table (hypergeometric test), accuracy-weighted mode
Ecosystem	Co-abundance network (4 layouts), taxonomy/module/enrichment coloring, Louvain communities, FBM overlay
Stability	Kuncheva/Tanimoto/CW_rel dual-axis chart, hierarchical clustering heatmap, feature presence across sparsity
Basket	Bookmarked models table, metrics comparison bars, feature overlap, coefficient heatmap
Console	Live job log, timing flamegraph (debug mode)

Plotly.js Integration

All charts use **Plotly.js** (lazy-loaded for performance) with consistent theming via the `useChartTheme` composable. Charts are rendered with `{ responsive: true, displayModeBar: false }` and support dark/light mode switching via a `watch` on `themeStore.isDark`.

Dark/Light Theming

The `chartColors()` function returns a theme-aware color palette with properties: `text`, `grid`, `paper`, `plot`, `positive` (deepskyblue), `negative`

(firebrick), `neutral` (darkorchid), and alpha variants. The `chartLayout()` function applies these colors to a Plotly layout object.

10. Deployment & Configuration

Docker

Multi-stage Dockerfile:

```
# Stage 1: Build frontend (Node.js)
FROM node:20-alpine AS frontend-build
WORKDIR /app/frontend
RUN npm ci && npm run build

# Stage 2: Backend (Python)
FROM python:3.11-slim
COPY --from=frontend-build /app/frontend/dist /app/static
RUN pip install -e .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Kubernetes (Helm)

Helm charts in `helm/` provide production Kubernetes deployment with: configurable replicas, resource limits, PostgreSQL StatefulSet, Ingress with TLS, PersistentVolumeClaim for data storage, and health check probes.

Environment Variables

Variable	Default	Description
`DATABASE_URL`	sqlite+aiosqlite:///./predomics.db	Database connection string
`SECRET_KEY`	(required)	JWT signing key (256-bit)
`PROJECT_DIR`	./data/projects	Project file storage directory
`ALLOWED_ORIGINS`	http://localhost:5173	CORS allowed origins
`SMTP_HOST`	(optional)	SMTP server for email notifications
`SCITQ_SERVER`	(optional)	SciTQ server URL for cluster execution
`MAX_UPLOAD_SIZE`	500MB	Maximum file upload size

11. Testing

Backend Tests (pytest)

215+ test cases using **pytest** and **pytest-asyncio** with an in-memory SQLite database. Tests cover API endpoints, authentication flows, authorization checks, data processing, export generation, and error handling. Run with:

```
cd backend && python -m pytest --cov=app --cov-report=term-missing
```

Frontend Tests (Vitest)

Unit tests using **Vitest** with jsdom for DOM simulation. Tests cover component rendering, Pinia store logic, router guard behavior, and parameter validation. Run with:

```
cd frontend && npx vitest run --coverage
```


© 2024–2026 Edi Prifti & GMT Science. All rights reserved.